

LAB TASK # 9

Mian Daniyal Hassan

SAP ID # 70150006 [Company address]

```
.model smal
```

```
.stack 100h
```

```
.data
```

```
menu_msg db 13, 10, 'Choose operation:', 13, 10
```

```
db '1. Hex to Binary', 13, 10
```

```
db '2. Binary to Hex', 13, 10
```

```
db '3. Calculator', 13, 10
```

```
db 'Enter choice (1-3): $'
```

```
hex_prompt db 13, 10, 'Enter hex number (2 digits): $'
```

```
bin_prompt db 13, 10, 'Enter 8-bit binary number: $'
```

```
calc_prompt1 db 13, 10, 'Enter first number: $'
```

```
calc_prompt2 db 13, 10, 'Enter second number: $'
```

```
calc_menu db 13, 10, 'Enter 1 for Addition or 2 for Subtraction: $'
```

```
result_msg db 13, 10, 'Result: $'
```

```
newline db 13, 10, '$'
```

```
hex_input db 3 dup(0)
```

```
bin_input db 9 dup(0)
```

```
bin_result db 9 dup(0)
```

```
hex_result db 3 dup(0)
```

```
num1 dw ?
```

```
num2 dw ?
```

```
.code
```

```
main proc
```

```
mov ax, @data
```

```
mov ds, ax
```

; Display main menu

mov dx, offset menu_msg

mov ah, 9

int 21h

; Get choice

mov ah, 1

int 21h

cmp al, '1'

je hex_to_bin

cmp al, '2'

je bin_to_hex

cmp al, '3'

je calculator

jmp exit

hex_to_bin:

call hex_binary_conversion

jmp exit

bin_to_hex:

call binary_hex_conversion

jmp exit

calculator:

call calculator_proc

exit:

mov ah, 4ch

int 21h

main endp

; Procedure for Hex to Binary conversion

hex_binary_conversion proc

; Get hex input

mov dx, offset hex_prompt

mov ah, 9

int 21h

mov si, offset hex_input

mov cx, 2

read_hex:

mov ah, 1

int 21h

mov [si], al

inc si

loop read_hex

; Convert and display

mov si, offset hex_input

mov di, offset bin_result

mov cx, 2

convert_hex_loop:

mov al, [si]

cmp al, '9'

```

    jbe decimal_digit
    sub al, 'A'
    add al, 10
    jmp process_digit
decimal_digit:
    sub al, '0'
process_digit:
    mov bl, 4
convert_bits:
    shl al, 1
    mov dl, '0'
    jnc store_bit
    mov dl, '1'
store_bit:
    mov [di], dl
    inc di
    dec bl
    jnz convert_bits
inc si
loop convert_hex_loop

; Display result
mov dx, offset result_msg
mov ah, 9
int 21h
mov dx, offset bin_result
int 21h
ret
hex_binary_conversion endp

```

; Procedure for Binary to Hex conversion

binary_hex_conversion proc

; Get binary input

mov dx, offset bin_prompt

mov ah, 9

int 21h

mov si, offset bin_input

mov cx, 8

read_bin:

mov ah, 1

int 21h

mov [si], al

inc si

loop read_bin

; Convert and display

mov si, offset bin_input

mov di, offset hex_result

mov cx, 2

convert_bin_loop:

mov bl, 4

mov al, 0

convert_bin_bits:

shl al, 1

mov dl, [si]

cmp dl, '1'

```

    jne skip_add
    inc al
skip_add:
    inc si
    dec bl
    jnz convert_bin_bits

    cmp al, 9
    jbe bin_decimal_digit
    add al, 'A'-10
    jmp store_hex_digit
bin_decimal_digit:
    add al, '0'
store_hex_digit:
    mov [di], al
    inc di
    loop convert_bin_loop

; Display result
    mov dx, offset result_msg
    mov ah, 9
    int 21h
    mov dx, offset hex_result
    int 21h
    ret
binary_hex_conversion endp

; Calculator procedure
calculator_proc proc

```

```
; Input first number
mov dx, offset calc_prompt1
mov ah, 9
int 21h
call read_num
mov num1, ax
```

```
; Input second number
mov dx, offset calc_prompt2
mov ah, 9
int 21h
call read_num
mov num2, ax
```

```
; Show operation menu
mov dx, offset calc_menu
mov ah, 9
int 21h
```

```
mov ah, 1
int 21h
```

```
cmp al, '1'
je do_addition
cmp al, '2'
je do_subtraction
ret
```

```
do_addition:
```



```
mov ax, num1
add ax, num2
jmp show_calc_result
```

do_subtraction:

```
mov ax, num1
sub ax, num2
```

show_calc_result:

```
push ax
mov dx, offset result_msg
mov ah, 9
int 21h
pop ax
call display_num
ret
```

calculator_proc endp

; Procedure to read number from keyboard

read_num proc

```
push cx
mov cx, 0
```

read_loop:

```
mov ah, 1
int 21h
cmp al, 13
je read_done
sub al, '0'
```

```
mov cl, al
mov ax, 10
mul cx
add ax, cx
jmp read_loop
```

```
read_done:
    pop cx
    ret
read_num endp
```

; Procedure to display number in AX

```
display_num proc
```

```
    push ax
    push bx
    push cx
    push dx
```

```
    mov bx, 10
    mov cx, 0
```

```
divide_loop:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne divide_loop
```

display_loop:

pop dx

add dl, '0'

mov ah, 2

int 21h

loop display_loop

pop dx

pop cx

pop bx

pop ax

ret

display_num endp

end main

